

Big Data Computing

Written test 18/06/2025
(SOLUTIONS of PART 2)

Exercise 1 Let G be a $t \times t$ grid of *cells* covering a region of a mobile network, and let P be a dataset of N *connections* to the cells of G . Each connection is represented by a pair $(k, (i, j))$ where $k \in [0, N - 1]$ is a distinct integer key, and (i, j) , with $0 \leq i, j < t$, are the row and column indices of the cell used by the connection. Assume $t = O(\sqrt{N})$.

1. Design a **3-round** MapReduce algorithm that returns the rows of G such that more than $t/2$ of their cells have been used by connections of P . For each such row i , the algorithm must return a pair (i, t_i) , where $t_i > t/2$ is the number of distinct cells of row i were used by connections of P . No assumptions must be made on the number of connections per cell.
2. Analyze the local and aggregate space, M_L and M_A , of the algorithm. For full score, you must get $M_L = o(N)$ and $M_A = O(N)$.

Solution.

1. The MapReduce algorithm is the following.

Round 1:

- *Map phase*: map each pair $(k, (i, j))$, into the intermediate pair $(k \bmod \sqrt{N}, (i, j))$.
- *Reduce phase*: for each $0 \leq h < \sqrt{N}$, let L_h be the list of values (i.e., cell identifiers (i, j)) from the intermediate pairs with key h . For each (i, j) occurring in L_h return as output one pair $((i, j), 0)$, where (i, j) acts as key.

Round 2:

- *Map phase*: empty.
- *Reduce phase*: for each key (i, j) let $L_{i,j}$ be the list of at most $\sqrt{N}$ values (i.e., 0's) of output pairs of Round 1 with key (i, j) , and return the pair (i, j) where i acts as key.

Note that at this point, for every row i we have exactly one pair (i, j) for every distinct column j such that cell (i, j) was used by connections of P .

Round 3:

- *Map phase*: empty.
- *Reduce phase*: for each $0 \leq i < t$ let L_i be the list of values (i.e., distinct column indices j) of the output pairs of Round 2. For what we observed above, $|L_i| = t_i$. If $|L_i| > t/2$ return $(i, |L_i|)$, otherwise return nothing.

2. Let us analyze the local space first. In the map phase of Round 1, each application of the map function requires constant space; in the reduce phase of Rounds 1 and 2 each reducer processes and produces in output at most $\sqrt{N}$ pairs; in the reduce

phase of Round 3, each reducer processes at most t pairs and produces at most one pair. Therefore, $M_L = O(\sqrt{N} + t)$ which is $O(\sqrt{N})$, since we assumed $tO(\sqrt{N})$. The aggregate space is $M_A = O(N)$ since, as it is easily seen, the number of input, intermediate, and output pairs of each round is $O(N)$.

□

Exercise 2 Let P be a set of n D -dimensional Boolean vectors. Suppose that P is stored into a hash table T built using a hash function h randomly drawn from the bit-sampling LSH family $\mathcal{H} = \{h_i : 0 \leq i < D\}$. For $h \in \mathcal{H}$, let " $\text{not}(h(x))$ " denote the negation of the binary value $h(x)$.

1. Given two vectors p and q , determine the probability $\Pr_{h \in \mathcal{H}}[h(p) = \text{not}(h(q))]$ as a function of the Hamming distance $d_H(p, q)$.
2. Given a query vector q , we want to find an r -far vector $p \in P$, i.e., such that $d_H(p, q) \geq r$. Based on the above analysis, how would you efficiently search such a p in the table T ? What probabilistic guarantees does your method provide?

Solution.

1. For $0 \leq i < D$, let p_i and q_i denote the i -th bit of p and q , respectively. It is clear that

$$h_i(p) = \text{not}(h_i(q)) \Leftrightarrow p_i \neq q_i.$$

Therefore,

$$\Pr_{h \in \mathcal{H}}[h(p) = \text{not}(h(q))] = \frac{d_H(p, q)}{D}.$$

2. Given a query vector q one can scan bucket $T[\text{not}(h(q))]$ returning the first vector p with $d_H(p, q) \geq r$, if one exists, or null otherwise. If there exists an r -far vector $x \in P$, the probability of returning a correct solution is at least the probability that x ends up in bucket $T[\text{not}(h(q))]$. Based on the previous analysis, this probability is

$$\Pr_{h \in \mathcal{H}}[h(x) = \text{not}(h(q))] \geq \frac{r}{D}.$$

□